

Reporte Hormigas de Langton

Eduardo Hernández Castro

Clase Complex Sytems – Profesor GENARO JUAREZ MARTINEZ –
Grupo 7CM4

1 Introducción

En este documento se registran las observaciones y fundamentos técnicos de la simulación de hormigas basada en las reglas de Langton y el comportamiento de los distintos tipos de hormigas.

2 Fundamento del programa

La simulación está diseñada para evaluar cómo interactúan cuatro tipos de hormigas en una grilla con reglas de movimiento de Langton y ciclo de vida controlado. Cada sección del código aporta una parte clave del modelo.

2.1 Reglas principales

- Cada hormiga vive hasta 80 iteraciones. Esta regla se define en `config.py` mediante la constante `LIFESPAN` y se aplica en `simulation.py` durante la fase de envejecimiento y muerte.
- El estado de cada celda es negro/blanco, pero ese estado se maneja internamente y no se muestra en la visualización. El estado oculto se representa en `grid.py` con el arreglo `cell_state`.
- En celda negra la hormiga gira a la izquierda; en celda blanca gira a la derecha. Esta lógica está en `grid.py` dentro de la función `apply_langton_rule`.
- Después de girar, la hormiga invierte el color de la celda y avanza una celda. El volteo de la celda se hace con `flip_cell_state` en `grid.py` y el movimiento con `move_ant` o `attempt_movement_with_collision`.
- Si la celda destino está ocupada, la hormiga prueba hasta tres direcciones alternativas de forma aleatoria. Esta reintención se gestiona en `grid.py` dentro de `attempt_movement_with_collision`.
- Si todas las direcciones libres están ocupadas, la hormiga espera y no se mueve en esa iteración. El ant permanece en su posición cuando `attempt_movement_with_collision` no encuentra destino libre.

- Una hormiga nueva nace cuando una reproductora y una reina se enfrentan en direcciones opuestas y se cumple la probabilidad de reproducción. Esta interacción se evalúa en `simulation.py` durante la fase de reproducción, usando `REPRODUCTION_PROBABILITY` y `OPPOSITE_DIRECTIONS` desde `config.py`.
- Se detectan escenarios de aislamiento, sobrepoblación y estabilidad con umbrales de población y ocupación. Estas condiciones se definen en `config.py` y se usan en `simulation.py` dentro de `detect_scenariio`.

3 Estructura del código

La aplicación está organizada en módulos claros que separan la lógica de configuración, el modelo de datos, la simulación y la visualización.

3.1 Configuración

El archivo `config.py` contiene las constantes que determinan el comportamiento general:

- `GRID_SIZE`, `OCCUPANCY_RATIO` y `MAX_ANTS` para la dimensión y el número de hormigas iniciales.
- `ANT_DENSITIES` para la distribución inicial de los tipos de hormiga: 1% reinas, 55% trabajadoras, 9% reproductoras y 35% soldados.
- `LIFESPAN` define las 80 iteraciones de vida máxima de cada hormiga.
- `REPRODUCTION_PROBABILITY` y `OPPOSITE_DIRECTIONS` controlan cuándo se puede generar una nueva hormiga.
- `ISOLATION_THRESHOLD`, `OVERPOPULATION_GRID_THRESHOLD`, `OVERPOPULATION_DEATH_RATE`, `STABLE_POPULATION_VARIANCE` y `STABLE_GENERATIONS_REQUIRED` fijan los umbrales de escenario.

3.2 Modelo de hormigas

El módulo `ant.py` define la clase base `Ant` y las subclases `Queen`, `Worker`, `Reproducer` y `Soldier`. Cada hormiga mantiene su posición, dirección, edad y color de visualización. El método `age_one_iteration()` incrementa la edad y `is_alive()` determina si la hormiga debe morir cuando supera la longevidad.

3.3 Grilla y reglas de Langton

El módulo `grid.py` implementa la grilla de ocupación y el estado oculto de las celdas. Los componentes más relevantes son:

- `occupancy`: matriz booleana que indica qué celdas tienen hormigas.
- `cell_state`: matriz de valores 0/1 que representa el color oculto de cada celda.
- `apply_langton_rule`: aplica la regla de giro y calcula la nueva posición.

- `flip_cell_state`: cambia el color de la celda actual después del giro.
- `move_ant`: mueve la hormiga a una celda vacía.
- `attempt_movement_with_collision`: gestiona choques y reintentos en hasta tres direcciones adicionales.
- `get_occupancy_ratio`: calcula el porcentaje de celdas ocupadas por hormigas.

3.4 Simulación

El módulo `simulation.py` coordina la ejecución completa:

- `_initialize_ants`: crea la configuración inicial, asignando posiciones aleatorias y tipos de hormiga según `ANT_DENSITIES`.
- `iterate`: ejecuta el ciclo de movimiento, reproducción, envejecimiento y muerte por generación.
- `detect_scenario`: clasifica el estado actual como aislamiento, sobrepoblación o estabilidad.
- `run`: itera el número especificado de generaciones.
- Estadísticas por generación: total de hormigas, conteos por tipo, nacimientos, muertes, edad promedio y ocupación.

3.5 Visualización y reporte

El módulo `visualization.py` produce la salida gráfica y los resúmenes:

- Gráficos de evolución poblacional y ocupación.
- Guardado de imágenes iniciales y finales del grid.
- Exportación de un archivo de resumen `summary.txt` con las métricas clave de cada corrida.

3.6 Entrada principal

El archivo `main.py` expone el CLI para ejecutar la simulación con parámetros ajustables:

- número de iteraciones,
- tamaño de la grilla,
- ocupación inicial,
- modo toroidal,
- generación de reportes y CSV,
- registro de salida en archivo.

4 Escenarios

Según el código en `simulation.py` y los umbrales definidos en `config.py`, los escenarios se clasifican así:

1. Aislamiento: ocurre cuando la población actual baja por debajo del 10% de la población inicial. En nuestro caso, con 5000 hormigas iniciales, aislamiento se detecta si quedan menos de 500 hormigas.
2. Sobrepoblación: ocurre cuando la ocupación de la grilla excede 80% y la tasa de muerte de la iteración es mayor al 15%. Esto implica una saturación del espacio combinada con un aumento rápido de defunciones.
3. Sistema estable: ocurre cuando el total de hormigas se mantiene con una varianza relativa de 5% o menos durante al menos 50 generaciones consecutivas.

5 Resultados

En la carpeta `output` se generaron los resultados de las corridas estudiadas.

5.1 Caso 1 – Seed 29976446, 100x100, 200 generaciones

- Archivo de resumen: `S29976446_G100_I200_00.5_toroidal_20260605_074107_summary.txt`
- Generaciones: 200
- Hormigas finales: 1 (soldado)
- Ocupación final: 0.01%
- Hormigas en generación 80: 147
- Escenario detectado: aislamiento

5.2 Caso 2 – Seed 401529739, 100x100, 250 generaciones

- Archivo de resumen: `S401529739_G100_I250_00.5_toroidal_20260605_075905_summary.txt`
- Generaciones: 250
- Hormigas finales: 1 (trabajadora)
- Ocupación final: 0.01%
- Hormigas en generación 80: 183
- Escenario detectado: aislamiento

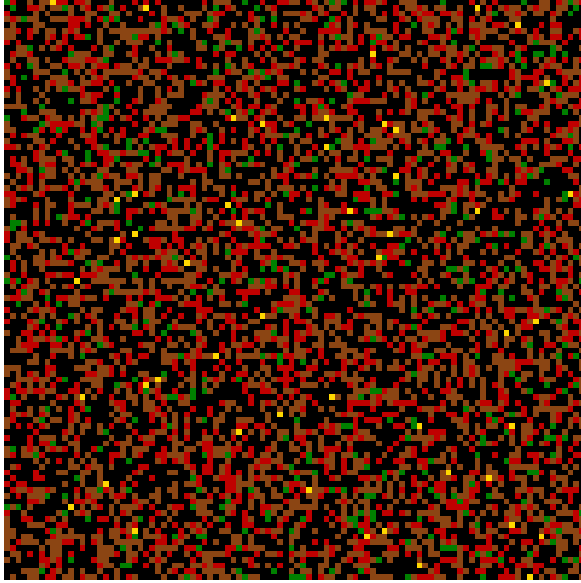


Figure 1: *
Caso 1: estado inicial

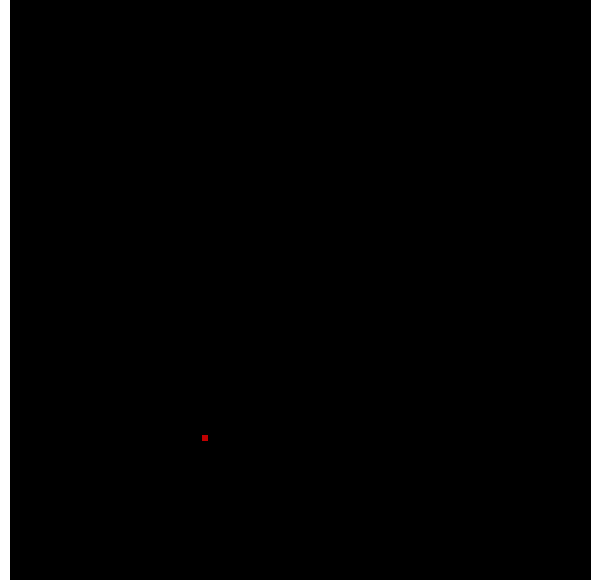


Figure 2: *
Caso 1: estado final

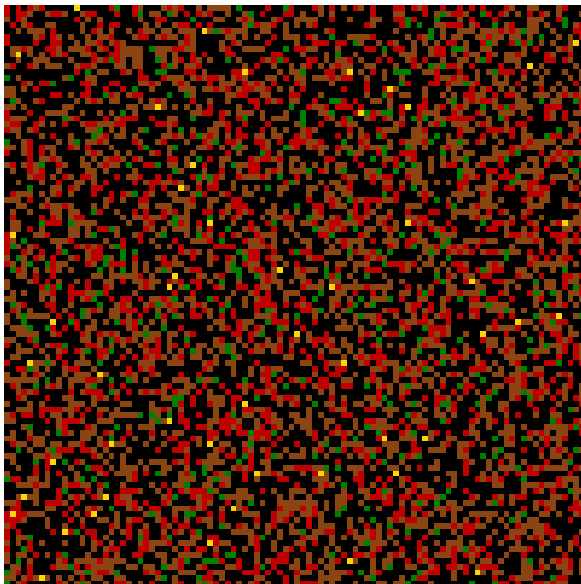


Figure 3: *
Caso 2: estado inicial

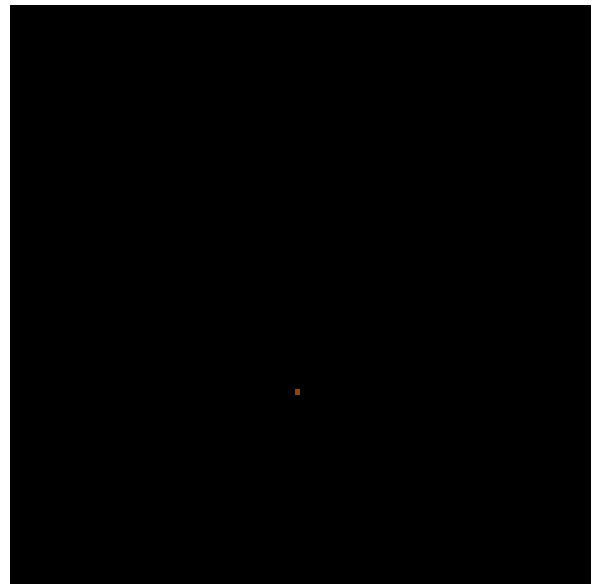


Figure 4: *
Caso 2: estado final

5.3 Notas de terminal y logs

El log `output/logs/find_seed_20260605_074103.txt` confirma la búsqueda de semilla y los resultados:

- Grid inicial 100x100 con 5000 hormigas y 50% de ocupación.
- Al finalizar 200 iteraciones quedó 1 hormiga y 0.01% de ocupación.
- En iteración 80 se tenía 147 hormigas.

5.4 Dependencia de la semilla y el tamaño del grid

La configuración inicial depende de la semilla aleatoria y del tamaño del grid. Al ser la grilla toroidal, dos tamaños de grid diferentes no pueden producir la misma configuración inicial válida, ya que la estructura de vecinos y la cantidad de celdas difieren.

6 Conclusiones

Las observaciones principales son:

- La evolución del sistema depende fuertemente del grid elegido y de la semilla aleatoria.
- Con un grid de 100x100 y 50% de ocupación se generan 5000 hormigas iniciales, lo que da lugar a un espacio de estados iniciales extremadamente grande: elegir 5000 posiciones entre 10000 disponibles y asignar tipos de hormiga según proporciones de 1%, 9%, 35% y 55%.
- Existe la posibilidad de que dos semillas diferentes produzcan el mismo estado inicial, pero solo si coinciden tanto las posiciones ocupadas como los tipos asignados. En la práctica, dado el gran número de combinaciones, esto es muy poco probable.
- Los resultados de las pruebas muestran que, en estos casos, el sistema termina en aislamiento: la mayoría de las hormigas mueren y quedan 1 hormiga después de 200 o 250 generaciones.
- La carpeta `output/logs` aporta evidencia del comportamiento en iteración 80 y de la ocupación final, confirmando la detección de aislamiento.
- Se debe seguir investigando para encontrar configuraciones que mantengan hormigas vivas más allá de 250 generaciones y para comparar con los escenarios de sobrepoblación y estabilidad.

7 Próximos pasos

- Analizar múltiples ejecuciones con semillas diferentes y comparar las trayectorias.
- Comparar resultados de los escenarios de aislamiento, sobrepoblación y estabilidad.
- Ajustar probabilidades de reproducción y densidades iniciales para evaluar su efecto en la supervivencia.
- Agregar las imágenes de `test_run2` en el informe una vez que estén disponibles.
- Registrar y comparar los resúmenes de los archivos `summary.txt` generados.